

Validating Black Box Neural Networks

Dale Chant Red Centre Software Pty Ltd
Michael Potter Red Centre Software Pty Ltd

International Journal of
Market Research
2019, Vol. 61(3) 338–344
© The Author(s) 2019
Article reuse guidelines:
sagepub.com/journals-permissions
DOI: 10.1177/1470785319825977
journals.sagepub.com/home/mre



**Association of Survey Computing (ASC)
conference, 'Alexa, what's the future of
market research?', 15th November 2018,
Ort House, London**

Abstract

Machine learning, driven by recent advances in neural net technology, holds much promise, but how to validate any particular model? This note looks at why validation is necessary, and describes some practical techniques for doing so. The example scenario (from a Health Insurance survey, $N=29,145$) is a sentiment analysis of verbatims arising from the Net Promoter Score expatiate 'Why that rating?' We show that respondents' ratings can be predicted to a useful level of accuracy, and that the model can be validated by reference to external benchmarks.

Keywords

machine learning, Net Promoter Score, neural nets, NPS, sentiment analysis

Why validate?

Neural nets are practically opaque to a debug walk-through. Reruns against the same training data can yield quite different results because network layers are initialised by random weights and biases, and thus there is no guarantee that the cost function has not found misleading local minima.

Sanderson (n.d.) shows that visualisations of the inner layers of a hand-written digit recognition model bear no evolving relationships to anything a human would recognise. His simple model operates in a 13,002-dimensional space (input pixels*layer weights and biases), well beyond our ability to fathom.

Therefore, one cannot explain why or how a particular prediction was correct or incorrect.

Corresponding author:

Dale Chant, Red Centre Software Pty Ltd, Melbourne, VIC 3000, Australia.
Email: dale@redcentresoftware.com

Table 1. Frequency counts of each rating.

Q4a likely to recommend											
Rating	0	1	2	3	4	5	6	7	8	9	10
Frequency	773	238	376	567	610	3,690	2,684	4,867	6,033	3,745	5,562

Approaches to validation

Indirect methods are required. First, ensure the model is internally consistent, and then compare with independent benchmarks.

To determine consistency, test the model on its own training data. Having already seen the answers, how well does it remember? Next, compare multiple runs, where disparities measure instability. Then check that the ratio of correct:total degrades naturally as resolution increases. One would expect binary sentiment (predict just positive or negative) to be more accurate than predicting a value from 0 to 10.

Since a model's outputs cannot be traced from the inputs, the next recourse is benchmarking against procedures or algorithms, which can be formally verified.

For hand-written digits, the output should be at least better than a random guess (a 1 in 10 chance of being correct). Measuring darkness, where a '1' has fewer pixels than an '8', and similar approaches, can give up to 50% correct (Nielsen, 2015).

For Brand Coding, compare the correct rate with that obtained from human coders or approximate string-matching algorithms such as Damerau–Levenshtein (Levenshtein, 1965; Ukkonen, 1983).

For text sentiment, compare the correct rating with that obtained from the R *Syuzhet* package (Jockers, 2017), which sums predefined word weights.

For all, run correlations of trained versus predicted on the same case set, looking for a score of at least 0.9.

Predicting likely to recommend (NPS) from sentiment

Consider the standard NPS questions:

- Q4a: On a scale of 0–10, how likely are you to recommend <brand> to friend or relative? Answer '8'
- Q4b: Why did you give that rating? Answer 'Good service, reasonable price'

Rating + expatiate questions provide ready-to-go self-tagged training sets. This respondent equates the sentiment of 'Good service, reasonable price' to be 8. Given just the text, how well can the sentiment score of 8 be predicted?

There are generic verbatim issues which inevitably complicate matters: truncation of long responses, slang and abbreviations, poor formatting and missing spaces, general nonsense, obscenities, neologisms, skimmers (random characters) and wildly unorthodox spellings. Such cases should be removed from consideration, since noise will degrade both benchmark and validation.

The Q4a distribution is given in Table 1.

Over 90% of the 29,145 respondents gave a rating ≥ 5 . By manual verbatim review, the negative/positive apex is between 6 and 7, and disengaged respondents cluster at ratings 0–4.

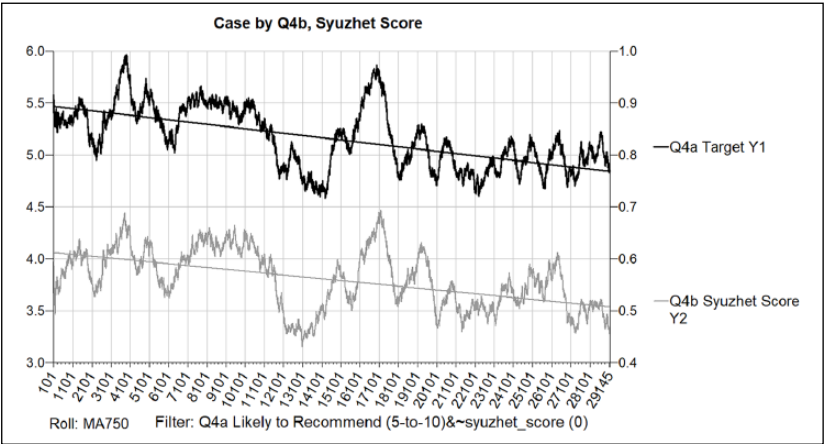


Figure 1. X-axis is one case per point. Black is the target, being the respondents' own ratings. Grey is the *Syuzhet* score on verbatims.

Table 2. Resolution nettings.

Target code	Res2	Res3 (NPS)	Res4	Res5	Res I I
0	Q4a(1/6)	—	—	—	—
1	Q4a(7/10)	Q4a(1/6)	Q4a(0/3)	Q4a(0/2)	Q4a(1)
2	—	Q4a(7/8)	Q4a(4/6)	Q4a(3/4)	Q4a(2)
3	—	Q4a(9/10)	Q4a(7/8)	Q4a(5/6)	Q4a(3)
4	—	—	Q4a(9/10)	Q4a(7/8)	Q4a(4)
5	—	—	—	Q4a(9/10)	Q4a(5)
—	—	—	—	—	—
10	—	—	—	—	Q4a(10)

Independent benchmark: *Syuzhet* scores

Syuzhet is an R package for text analysis. Sentiment is evaluated by reference to a dictionary of weighted words. For example, if love=0.75, beauty=0.5 and good=0.5 then ‘Love of beauty is good’ scores $0.75 + 0.5 + 0.5 = 1.75$

The most negative response scores –4:

... lack of details concerning claimable areas ... non-claimable costs ... feels cheap and nasty to say a health service is claimable, only to discover it's not ... disappointing and covert ...

Most positive response scores 8.4:

... The yearly cost was half ... the benefits were immensely better ... the best value available today ... not for profit fund ... accessible for helpful information ...

Noise cases are classified as *Syuzhet* score=0 or Q4a less than 5. If there is not enough sentiment for a non-zero *Syuzhet* score then machine learning will also have difficulties. Ratings 0–4 comprise a 10% subset with many low engagement verbatims.

The *Syuzhet* benchmark, de-noised by case filter and moving average, is remarkably good (Figure 1).

Can machine learning improve on *Syuzhet*?

We used the new Microsoft (n.d.-b) ML.Net suite. This is free and easy to deploy on Windows machines, with programming in Visual Basic. Adapting the supplied examples was not difficult.

For the model inputs, we assembled text files at each target resolution formatted as < target rating > tab < verbatim >. The targets are defined as nets of the respondent Q4a ratings (Table 2).

The model was executed five times at each resolution. The key steps are as follows:

```
For runID = 1 To 5
    model = TrainSentimentMultiClass("Q4a_1", InputTextVar)
    EvaluateSentimentMultiClass(model, InputTextVar, runID)
Next

Public Function TrainSentimentMultiClass(...)
    ...
    Dim sdcac As New
    Trainers.StochasticDualCoordinateAscentClassifier
    ...
    model = pipeline.Train(Of ClassificationData,
    ClassPredictionMultiClass)

    Return model
End Function

Public Function EvaluateSentimentMultiClass(model...)
    ...
    pred = model.Predict(verbatim)
    ...
End Function
```

The black box is *StochasticDualCoordinateAscentClassifier* (Microsoft, n.d.-a). The ML.Net suite has about 20 algorithms. This was chosen empirically on the basis that it gave the best results.

The performance of each run at each resolution is quantified as the ratio of correct:total, calculated as $100 * (\sum \text{corrects}) / (\sum \text{bases})$, for example, at Resolution 2, Run 1 (Table 3).

The results are given in Table 4.

The ratios at each resolution are stable across all runs, varying only by tenths of a percent.

Resolutions 3, 4 and 5 degrade naturally, from an average of 78.87% down to 72.98%. Resolution 2 (binary) at 96.3% is the best, but it is the easiest of targets. The big drop to 46.47% for resolution 11 is because there are no further even subdivisions to 11 targets.

Is resolving to 11 points too ambitious? The model runs are on the full case data, including nonsense verbatim cases and verbatims with no measurable sentiment content. The *Syuzhet* plot was smoothed and filtered to *Syuzhet_score* not 0 and Q4a is 5–10, so for a fair comparison, we must here do likewise (Figure 2).

The runs are nearly identical, and all have better coherence with the target than the *Syuzhet* plot, particularly around case 12,500 and from cases 26,000 ff.

Table 3. Cross tabulation of predicted (banner) by target (stub). Correct ratio is 96.30%, calculated as $100 \times (8460 + 19607) / (9060 + 20085)$.

96.30% correct		Predicted Resolution 2 Run 1	
		Negative	Positive
Q4a Likely to Recommend	Cases	9,060	20,085
	Net 0–6	8,460	478
		93%	2%
	Net 7–10	600	19,607
		7%	98%

Table 4. Actual and average ratios for each run at each resolution.

	Runs					Avg
	1	2	3	4	5	
Res2	96.3	96.31	96.3	96.31	96.31	96.30
Res3	78.27	78.84	78.84	79.11	79.31	78.87
Res4	74.11	74.24	74	74.42	74.63	74.28
Res5	73.05	72.95	72.9	72.93	73.07	72.98
Res11	46.86	46.5	46.18	46.38	46.42	46.47

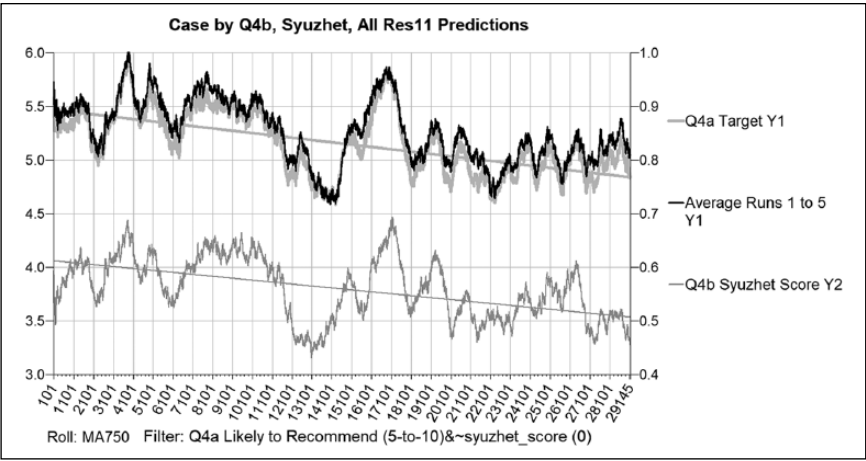


Figure 2. The fat grey series is the prediction target. The black series sitting on top is the average of the five runs at resolution 11.

Correlation

The final check for validity is to run a standard pairwise correlation on the aggregated verbatims. The input matrix is 17,275 rows of the form as given in Table 5.

Note the aggregation of common verbatims such as ‘Better rebates’. The correlation matrix gives better than 0.99 for each run against Q4a, with average as 0.9924, and the correlation of the runs against each other as 0.9967 (Table 6).

Table 5. Per case predicted against actual (leftmost column).

	Q4a Target	Predicted at Resolution 11				
		Run1	Run2	Run3	Run4	Run5
100% refund	8	8	8	7	8	8
All good	8	9	10	9	10	9
Always give good service, no hassles	10	10	10	10	10	10
Always used them never had any problems	10	9	9	9	9	9
Best service, best fees, least gaps, best incentives	9	10	10	10	10	10
Better packages for all insurance and discounts for number of years (loyalty)	7	8	8	7	8	8
Better rates – less hassle	7	7	7	7	7	7
Better rebates	129	126	126	126	144	126

Table 6. Correlation of all runs against the target, and against each other.

	Q4a	Run 1	Run 2	Run 3	Run 4	Run 5
Q4a	1					
Run 1	0.993	1				
Run 2	0.993	0.997	1			
Run 3	0.992	0.997	0.996	1		
Run 4	0.992	0.996	0.998	0.997	1	
Run 5	0.992	0.996	0.997	0.996	0.997	1

Conclusion

The StochasticDualCoordinateAscentClassifier is a better predictor than *Syuzhet*.

Consistent correct ratios across runs.

Reasonable degradation with increasing resolution.

Excellent match on de-noised per-case plots.

De-noised correlation >0.99.

The model can therefore be deemed to be sufficiently valid to deploy against unseen cases.

Acknowledgements

The sub-title to the one-day conference was ‘The application of artificial intelligence and machine learning to surveys’. You will find reviews of this conference in my Editor’s blog published in January (<https://twitter.com/tweetijmr/status/1080447666709057538>) and in the Editorial in this issue. Some of the presentations from the day can be found at <http://www.asc.org.uk/events/november-2018>. This article is a text version of the presentation by Dale Chant ‘Validating Black Box Neural Networks’.

Funding

The author(s) received no financial support for the research, authorship and/or publication of this article.

References

- Jockers, M. (2017). *Introduction to the Syuzhet Package*. Retrieved from <https://cran.r-project.org/web/packages/syuzhet/vignettes/syuzhet-vignette.html>
- Levenshtein, V. I. (1965). Binary codes capable of correcting deletions, insertions and reversals. *Doklady Akademii Nauk*, 163, 845–848 (Reprinted from *Soviet Physics Doklady*, 10, 707–710, 1966).
- Microsoft. (n.d.-a). *Documentation and discussion*. Retrieved from <https://docs.microsoft.com/en-us/dotnet/api/microsoft.ml.legacy.trainers.stochasticdualcoordinateascentclassifier>
- Microsoft. (n.d.-b). *ML.Net is a new AI platform by Microsoft*. Retrieved from <https://dotnet.microsoft.com/apps/machinelearning-ai/ml-dotnet>
- Nielsen, M. A. (2015). *Neural networks and deep learning*. Determination Press, Chapter 1. Retrieved from <http://neuralnetworksanddeeplearning.com/chap1.html>
- Sanderson, G. (n.d.). *Gradient descent, how neural networks learn*. Retrieved from <https://www.youtube.com/c/3blue1brown>
- Ukkonen, E. (1983) On approximate string matching. In: Karpinski M. (eds) Foundations of Computation Theory. FCT 1983. Lecture Notes in Computer Science, vol 158. Springer, Berlin, Heidelberg. https://link.springer.com/chapter/10.1007%2F3-540-12689-9_129

Copyright of International Journal of Market Research is the property of Sage Publications Inc. and its content may not be copied or emailed to multiple sites or posted to a listserv without the copyright holder's express written permission. However, users may print, download, or email articles for individual use.